

ROS2 day1 hw2 레포트

로봇 예비단원 주호진

1. 과제

- Topic통신 구현

c++ 패키지 제작하여 publisher, subscriber 노드 작성

python 패키지 제작하여 publisher, subscriber 노드 작성

콜백 받은 데이터를 터미널에 출력

토픽 통신 시 int, string, float, bool 메시지 형태 모두 사용하여 최소 데이터 1개 이상 주고 받을 것

-하나의 패키지 내에서 2개의 노드를 통해 토픽 통신하는 과정

-std_msgs를 활용해 c++패키지와 python 패키지가 서로 토픽 통신하는 과정 위 두 과정 모두 구현할 것

rqt graph 실행 후 노드, 토픽 연결 구조 화면 캡처

보고서 작성 시 해당 캡처 사진에 대해 간단한 설명 첨부 필수

2. 코드

코드는 패키지 기본 세팅을 제외한 추가한 부분을 기재했다.

day1_hw2_pub_node.hpp && publisher.py

```
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/bool.hpp"
#include "std_msgs/msg/float32.hpp"
#include "std_msgs/msg/int32.hpp"

rclcpp::Publisher<std_msgs::msg::Int32>::SharedPtr pubInt_;
rclcpp::Publisher<std_msgs::msg::Bool>::SharedPtr pubBool_;
rclcpp::Publisher<std_msgs::msg::Float32>::SharedPtr pubFloat_;
```

Std_msgs를 활용하는 것이 조건이었고 int, string, float, bool을 모두 사용해야하기 때문에 해당 자료형으로 publisher할 수 있도록 선언했다.

day1_hw2_pub_node.cpp && publisher.py

```
publisher_ = this->create_publisher<std_msgs::msg::String>("topic_string", 10);
pubInt_ = this->create_publisher<std_msgs::msg::Int32>("topic_int", 10);
pubBool_ = this->create_publisher<std_msgs::msg::Bool>("topic_bool", 10);
pubFloat_ = this->create_publisher<std_msgs::msg::Float32>("topic_float", 10);
// 각각의 topic에 대해 각각의 topic_name, topic_type, qos_profile을 설정한다.
self.string_publisher = self.create_publisher(String, 'string', qos_profile)
self.int_publisher = self.create_publisher(Int32, 'int', qos_profile)
self.bool_publisher = self.create_publisher(Bool, 'bool', qos_profile)
self.float_publisher = self.create_publisher(Float32, 'float', qos_profile)

qos_profile = QoSProfile(depth=10)
```

Node 클래스에 들어있는 create_publisher를 사용해 자료형과, topic이름, *QoS를 설정한다.

*Quality of Service: 데이터 통신 네트워크상에 흐르는 데이터의 중요도를 분류하여, 이를 기반으로 우선순위를 부여하는 기능. QoS profile은 말 그대로 QoS를 설정하는 것이고 depth만 설정했기 때문에 depth를 제외한 나머지는 ros2에서 기본값으로 설정해놓은 값을 따른다.

Depth가 10이면 최근 10개의 메시지를 저장하겠다는 것이다. (신뢰성 중요도에 따라 값 변경)

```
auto msgInt = std_msgs::msg::Int32();
msgInt.data = 2024405020;
RCLCPP_INFO(this->get_logger(), "Publishing: '%d'", msgInt.data);
pubInt_->publish(msgInt);

auto msgBool = std_msgs::msg::Bool();
msgBool.data = true;
RCLCPP_INFO(this->get_logger(), "Publishing: '%d'", msgBool.data);
pubBool_->publish(msgBool);

auto msgFloat = std_msgs::msg::Float32();
msgFloat.data = 3.141592;
RCLCPP_INFO(this->get_logger(), "Publishing: '%f'", msgFloat.data);
pubFloat_->publish(msgFloat);

msgInt = Int32();
msgInt.data = 2024405020
self.int_publisher.publish(msgInt)
self.get_logger().info(f'Published message: {msgInt.data}')

msgBool = Bool()
msgBool.data = False
self.bool_publisher.publish(msgBool)
self.get_logger().info(f'Published message: {msgBool.data}')

msgFloat = Float32()
msgFloat.data = 3.141592
self.float_publisher.publish(msgFloat)
self.get_logger().info(f'published message: {msgFloat.data}')
```

콜백함수가 호출될 때 기본 예제에 있는 string 뿐만 아니라, int, bool, float 자료형도 보내도록 세팅해놓은 코드이다.

day1_hw2_sub_node.hpp && subscriber.py

```
#include "std_msgs/msg/float32.hpp"
#include "std_msgs/msg/int32.hpp"
#include "std_msgs/msg/string.hpp"

self.int_subscriber =
self.bool_subscriber =
self.float_subscriber =
```

```

void int_callback(const std_msgs::msg::Int32::SharedPtr msg) const;
rclcpp::Subscription<std_msgs::msg::Int32>::SharedPtr subInt_;

void bool_callback(const std_msgs::msg::Bool::SharedPtr msg) const;
rclcpp::Subscription<std_msgs::msg::Bool>::SharedPtr subBool_;

void float_callback(const std_msgs::msg::Float32::SharedPtr msg) const;
rclcpp::Subscription<std_msgs::msg::Float32>::SharedPtr subFloat_;

```

Publisher에서 오는 통신 자료형을 받아야 하기 때문에 동일한 자료형을 받을 수 있게 선언한다.

day1_hw2_sub_node.cpp && subscriber.py

```

subInt_ = this->create_subscription<std_msgs::msg::Int32>(
    "int", 10, std::bind(&Day1Hw2SubNode::int_callback, this, std::placeholders::_1));

subBool_ = this->create_subscription<std_msgs::msg::Bool>(
    "bool", 10, std::bind(&Day1Hw2SubNode::bool_callback, this, std::placeholders::_1));

subFloat_ = this->create_subscription<std_msgs::msg::Float32>(
    "float", 10, std::bind(&Day1Hw2SubNode::float_callback, this, std::placeholders::_1));

void Day1Hw2SubNode::int_callback(const std_msgs::msg::Int32::SharedPtr msg) const
{
    RCLCPP_INFO(this->get_logger(), "I heard Int: '%d'", msg->data);
}

void Day1Hw2SubNode::bool_callback(const std_msgs::msg::Bool::SharedPtr msg) const
{
    RCLCPP_INFO(this->get_logger(), "I heard Bool: '%d'", msg->data);
}

void Day1Hw2SubNode::float_callback(const std_msgs::msg::Float32::SharedPtr msg) const
{
    RCLCPP_INFO(this->get_logger(), "I heard Float: '%f'", msg->data);
}

self.int_subscriber = self.create_subscription(Int32, 'topic_int', self.subscribe_topic_message, qos_profile)
self.bool_subscriber = self.create_subscription(Bool, 'topic_bool', self.subscribe_topic_message, qos_profile)
self.float_subscriber = self.create_subscription(Float32, 'topic_float', self.subscribe_topic_message, qos_profile)

def subscribe_topic_message(self, msg):
    self.get_logger().info(f'Received message: {msg.data}')

```

Node 클래스에 들어있는 create_subscription를 사용해 자료형과, topic이름, QoS를 설정한다.

이때 자료형과 topic이름은 publisher와 동일한 형태여야 수신된다. (QoS 설정은 다르더라도 수신할 수 있다)

3. 작동 영상

[C++ to C++ 영상](#)

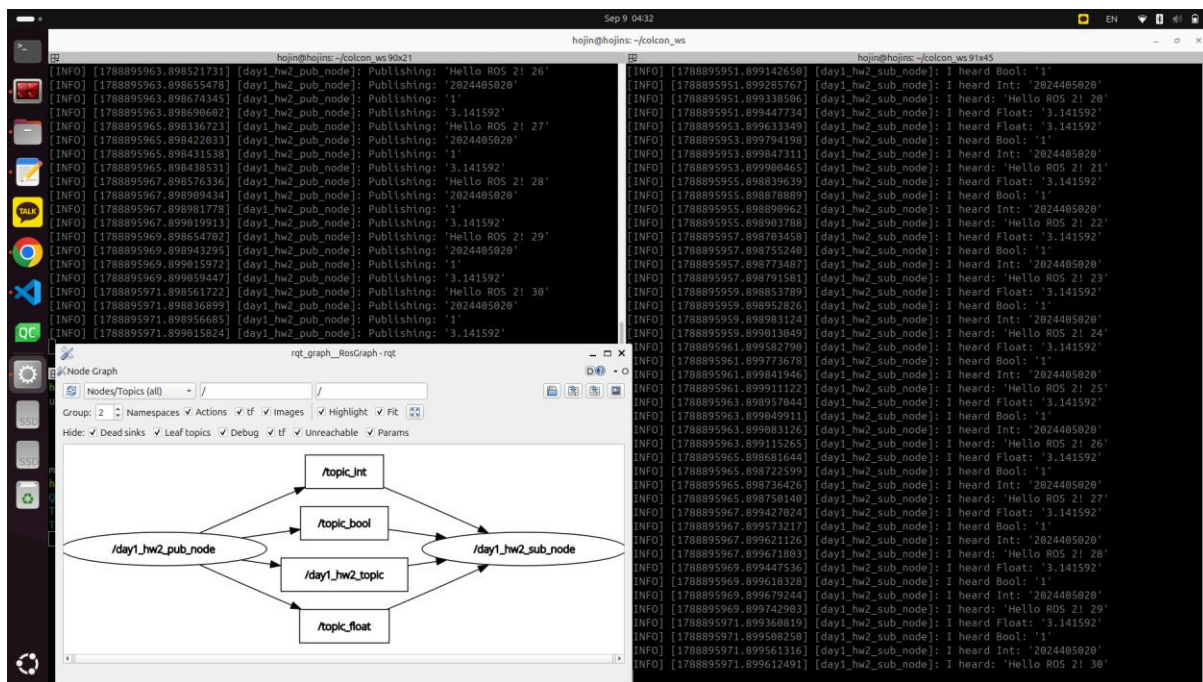
[Python to python 영상](#)

C++ to python 영상

python to C++ 영상

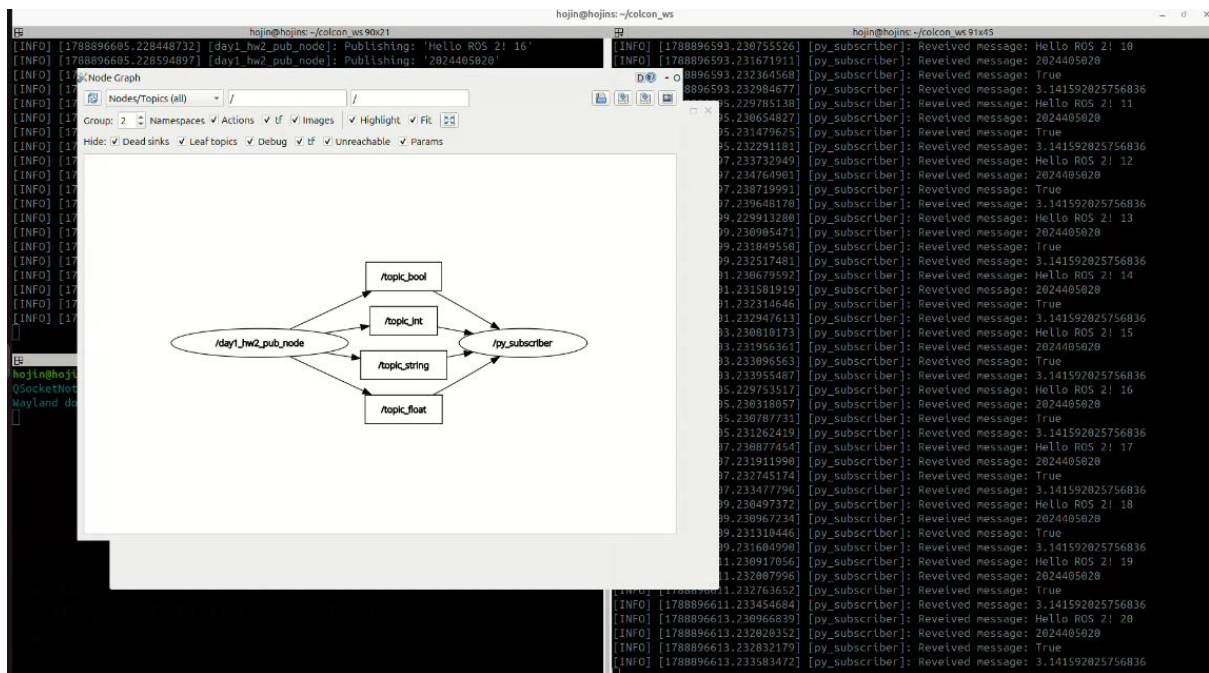
C++ to C++, py to py, C++ to py, py to C++ 모두 publisher에서 string, int, bool, float형을 동시에 보낸다.

C++ to C++



publisher의 topic 이름을 c++ subscriber에 설정해주었다.

Python to python



python to C++

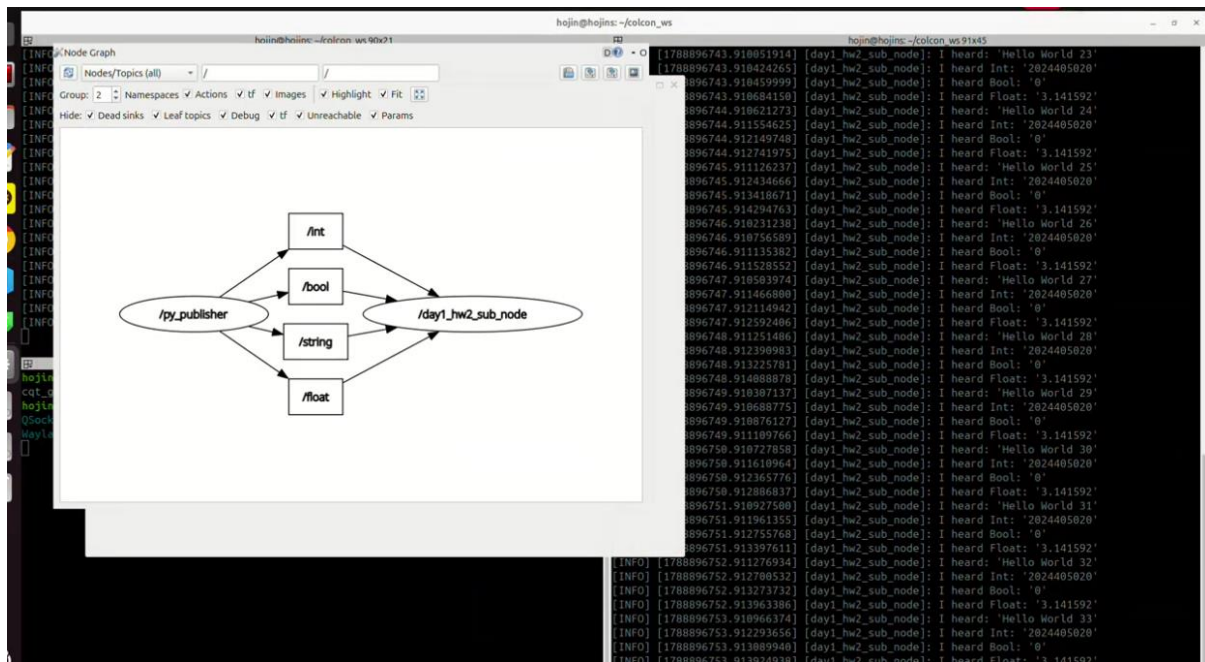
Python pub topic 이름을 C++에 연결

```
{
    subscriber_ = this->create_subscription<std_msgs::msg::String>(
        "string", 10, std::bind(&Day1Hw2SubNode::topic_callback, this, std::placeholders::_1));

    subInt_ = this->create_subscription<std_msgs::msg::Int32>(
        "int", 10, std::bind(&Day1Hw2SubNode::int_callback, this, std::placeholders::_1));

    subBool_ = this->create_subscription<std_msgs::msg::Bool>(
        "bool", 10, std::bind(&Day1Hw2SubNode::bool_callback, this, std::placeholders::_1));

    subFloat_ = this->create_subscription<std_msgs::msg::Float32>(
        "float", 10, std::bind(&Day1Hw2SubNode::float_callback, this, std::placeholders::_1));
}
```



4. AI 보고서

<https://share.gemini.google/a0ld0WcdCUdl>

<https://share.gemini.google/NwrzzIAOv5Yj>

패키지를 제작하고 설정하는 데에 많이 사용했다.

키지명과 동일한 폴더 애는 뭐야 include 폴더 안에 넣을 때 애는 무시하면 되나?

해당 폴더는 무시하면 안 되며, **헤더 파일(.hpp)** 을 관리할 때 반드시 사용해야 하는 필수 폴더입니다.

패키지 이름과 동일한 폴더가 존재하는 이유

- **이름 충돌 방지:** C++ 및 ROS 2 프로젝트에서 여러 패키지를 함께 사용할 때, 서로 다른 패키지의 헤더 파일 이름이 겹치는(예: 둘 다 `util.hpp` 를 가짐) 문제를 방지하기 위한 네임스페이스 역할을 합니다.
- **표준 포함 경로(Include Path):** ROS 2의 빌드 시스템(`ament_cmake`)은 `include/` 폴더 자체를 인클루드 디렉터리로 등록합니다. 따라서 하위에 패키지명 폴더가 있어야 패키지 간 구분이 가능해집니다.

가장 처음에 c++ 패키지를 만들었을 때 include 안에 패키지 명과 동일한 폴더가 하나

더 생겨서 질문하였다. 다른 ros2 패키지와 파일 이름이 겹칠 때 해당 폴더로 구분하는 역할을 한다는 것을 알게 되었다.

나는 왜 colcon_ws 폴더가 없는거지

colcon_ws 는 우분투나 ROS 2를 설치한다고 해서 **자동으로 생성되는 시스템 폴더가 아니기** 때문입니다. 사용자가 개발을 시작하기 위해 직접 작업 공간(Workspace) 이름을 정해서 생성해야 하는 수동 폴더입니다.

이러면 git으로 어떻게 관리하지 git으로 관리하는 폴더가 아닌데

Git으로 과제 코드를 깔끔하게 관리하면서 ROS 2 빌드도 동시에 하려면, 심볼릭 링크(바로가기)를 만들어 주는 것이 가장 좋습니다. 원본 코드는 Git 폴더 안에서 그대로 수정하고, colcon_ws/src/에서는 그 폴더를 바라보게 연결해 주는 방식입니다.

~/colcon_ws/src/ 여기에 링크만 두고 원본을 git 파일에서 관리해도 상관없나



네, 정확히 그렇게 하시는 것이 **가장 정석이고 실무에서도 강력히 추천하는 방식**입니다!

- **Git 관리:** 원본 코드는 원래 관리하시던 git 저장소(~/robit/21th_robit_intern_Hojin_Joo/...) 안에서 평소처럼 git add, git commit, git push 를 하시면 됩니다.

워크스페이스에 관하여 질문을 여러개 했다. 빌드 및 실행을 하려면 패키지가 워크스페이스 안의 src 폴더 하위에 위치해있어야 하는데, git으로 관리하는 과제 제출 폴더에 package를 만들어야 하기 때문에 워크 스페이스의 src에 심볼릭 링크를 연결하였다.

< > Home / colcon_ws / src



해당 노드는 해당 패키지 안에서만 있는건가



파일과 빌드 결과물 관점에서는 해당 패키지 안에 속하지만, 실행과 통신 관점에서는 다른 패키지와 완전히 자유롭게 상호작용할 수 있습니다.

파일(코드 및 실행 파일) 관점: 노드를 만드는 C++ 소스 파일, 빌드 설정(`CMakeLists.txt`), 그리고 빌드 후 생성되는 실제 실행 파일은 모두 그 패키지 폴더와 설치 경로(`install/.../lib/패키지이름/`) 내부에 묶여 관리됩니다.

Node 이름에 관해 질문을 여러개 했다. 만약 노드가 많아지는데 각기 다른 패키지에서 동일한 노드 이름을 가진다면 혼선이 오지 않을까라는 생각에서 질문하였고, 노드 이름이 길어지더라도 관리에 용이한 구조를 가지기 위해서는 직관적으로 알아볼 수 있는 이름으로 짓는 것이 맞겠다는 판단이 들었다.

```
source_scripts :  
'day1_hw2_py_pub_node  
'day1_hw2_py_sub_node Day1Hw2PubNode: Day1Hw2SubNode:
```